

Sintaxis básica en Java

Teoría para iniciar el bloque 1 del curso



 clase · main · llaves · punto y coma · salida · comentarios

Qué vamos a conseguir

Objetivos de aprendizaje del bloque de sintaxis

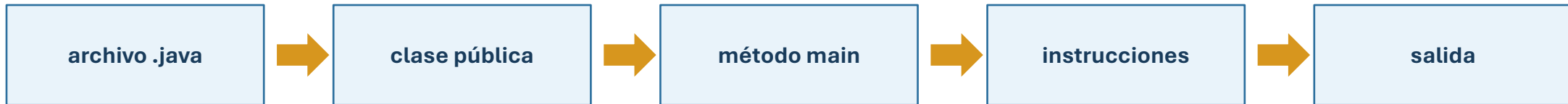
- Reconocer la estructura mínima de un programa Java.
- Entender qué es una clase y para qué sirve el método main.
- Usar correctamente llaves, paréntesis, comillas y punto y coma.
- Mostrar mensajes por pantalla con print y println.
- Localizar errores básicos antes de complicar el código.

Idea docente: que el alumnado pueda explicar cada línea antes de avanzar.

```
public class HolaMundo {  
    public static void main(String[] args) {  
        System.out.println("Hola, mundo");  
    }  
}
```

Mapa de la sintaxis básica

Las piezas que aparecen en casi todos los primeros programas



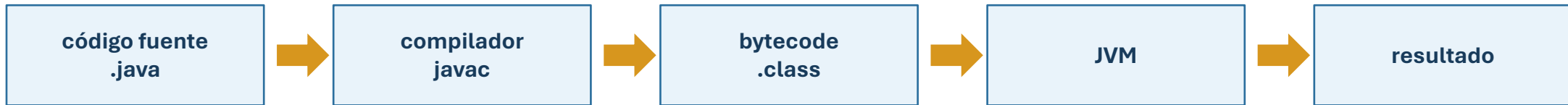
La sintaxis es el conjunto de reglas que Java necesita para entender nuestro código. Si falta una llave, una comilla o un punto y coma, el programa no puede compilar.

```
NombreArchivo.java → public class NombreArchivo
```

Java distingue mayúsculas y minúsculas: main no es Main, String no es string.

De .java a programa ejecutado

Qué ocurre cuando escribimos y ejecutamos código Java



- El archivo .java contiene el código que escribe el programador.
- javac revisa la sintaxis y genera un archivo .class.
- La JVM ejecuta el bytecode en el ordenador.
- Por eso primero compilamos y después ejecutamos.

```
javac HolaMundo.java  
java HolaMundo
```

Si javac falla, normalmente hay un error de escritura o falta configurar el JDK.

La clase: el contenedor del programa

En Java, casi todo vive dentro de una clase

Una clase es una estructura que agrupa código. En los primeros ejercicios solo la usamos como contenedor para poder ejecutar el método main.

```
public class HolaMundo {  
  
}
```

- **public**: indica que la clase es accesible.
- **class**: palabra reservada para declarar una clase.
- **HolaMundo**: nombre elegido para la clase.
- **{ }**: delimitan el bloque de código de la clase.

Clase pública y nombre del archivo

Una de las normas que más errores provoca al empezar

```
public class HolaMundo {  
    // código  
}
```

Si una clase es pública, el archivo debe llamarse exactamente igual que la clase, respetando mayúsculas y minúsculas.

Clase: HolaMundo
Archivo: HolaMundo.java

Incorrecto: holamundo.java, Hola mundo.java, HolaMundo.txt

Correcto: HolaMundo.java

El método main

El punto de entrada de una aplicación Java

```
public static void main(String[] args) {  
    // aquí empieza el programa  
}
```

Para nivel inicial no hace falta explicar aún public, static y void en profundidad. Basta con reconocerlos.

- Java busca este método para iniciar el programa.
- Debe escribirse main en minúscula.
- String empieza con S mayúscula.
- Los paréntesis contienen los argumentos.
- Las llaves contienen las instrucciones que se ejecutan.

Llaves, paréntesis y corchetes

Cada símbolo tiene una función concreta

```
public class Ejemplo {           // llave de clase

    public static void main(String[] args) { // método

        System.out.println("Hola");

    } // fin main
} // fin clase
```

- { } abren y cierran bloques.
- () contienen información de métodos o llamadas.
- [] aparecen en arrays, como String[] args.
- Cada apertura debe tener su cierre.

Si falta una llave, suelen aparecer errores al final del archivo.

El punto y coma

Marca el final de muchas instrucciones

En Java, muchas instrucciones terminan en punto y coma. Es como decirle al compilador: “esta orden ya ha terminado”.

```
System.out.println("Hola");  
int edad = 30;  
double precio = 12.50;
```

Error típico: `System.out.println("Hola")` → falta ;

- Después de una llamada a `println` sí suele ir ;
- Después de declarar una variable también.
- Después de abrir una clase o método no se pone ;

Instrucciones

Las órdenes que Java ejecuta una tras otra

Una instrucción es una orden concreta. Dentro del método main, Java ejecuta las instrucciones de arriba hacia abajo.

```
System.out.println("Primera línea");  
System.out.println("Segunda línea");  
System.out.println("Tercera línea");
```

- El orden importa.
- Cada línea puede mostrar algo, calcular o guardar un dato.
- Más adelante combinaremos instrucciones con variables y condiciones.

System.out.println()

La salida más usada al empezar

```
System.out.println("Hola, mundo");
```

Sirve para mostrar información por pantalla en la consola. El texto que queremos mostrar se escribe entre comillas dobles.

- System empieza por mayúscula.
- out va en minúscula.
- println escribe y baja a la línea siguiente.
- El texto literal va entre comillas dobles.

La salida se ve en la terminal, no directamente en el navegador.

print() y println()

La diferencia está en el salto de línea

```
System.out.print("Hola ");  
System.out.println("Patricia");  
System.out.println("Curso de Java");
```

print() deja el cursor en la misma línea. println() escribe el texto y después hace un salto de línea.

Resultado:
Hola Patricia
Curso de Java

Es útil para explicar que no todo System.out muestra igual.

Textos y comillas

Cómo escribir mensajes literales

```
System.out.println("Hola");      // correcto
System.out.println('Hola');      // incorrecto para texto
System.out.println("Edad: " + 30);
```

Ejemplo de carácter: char letra = 'A';

- Los textos largos se escriben con comillas dobles.
- Las comillas simples se usan para un solo carácter.
- El operador + permite unir texto y valores.
- Las comillas deben abrir y cerrar.

Comentarios de una línea

Ayudan a entender el código sin ejecutarse

```
// Este comentario explica la siguiente instrucción  
System.out.println("Estoy aprendiendo Java");
```

Todo lo que aparece después de // en esa misma línea es ignorado por Java. Sirve para explicar intención, avisos o pasos.

- No se ejecutan.
- No sustituyen al código claro.
- Deben explicar por qué se hace algo, no repetir lo evidente.

Comentarios de varias líneas

Útiles para enunciados y bloques de explicación

```
/*  
Enunciado:  
Crea un programa que muestre tres mensajes.  
Cada mensaje debe aparecer en una línea diferente.  
*/
```

El comentario empieza con `/*` y termina con `*/`. Todo lo que queda dentro se ignora al compilar.

En los ejercicios que estamos creando, el enunciado puede ir al principio dentro de este tipo de comentario.

Sangrado e indentación

No cambia el resultado, pero cambia la legibilidad

```
public class Ejemplo {  
    public static void main(String[] args) {  
        System.out.println("Hola");  
    }  
}
```

Java no obliga a sangrar para funcionar, pero un código bien alineado permite ver rápidamente qué está dentro de cada bloque.

- Clase: margen izquierdo.
- main: un nivel hacia dentro.
- Instrucciones: otro nivel hacia dentro.
- Llaves de cierre alineadas con su apertura.

Mayúsculas y minúsculas

Java es case-sensitive

En Java, cambiar una mayúscula por una minúscula puede hacer que el programa deje de funcionar.

```
public static void main(String[] args) {  
    System.out.println("Hola");  
}
```

Correcto: main, String, System, println y ; al final.

```
public static void Main(string[] args) {  
    system.out.println("Hola")  
}
```

Incorrecto: Main, string, system y falta de punto y coma.

Compilar desde terminal

Paso necesario para convertir .java en .class

Desde la carpeta donde está el archivo, usamos javac seguido del nombre completo del archivo.

```
javac HolaMundo.java
```

No se escribe java HolaMundo.java para ejecutar: eso es para compilar con javac.

- Si todo está bien, no suele aparecer ningún mensaje.
- Se crea un archivo HolaMundo.class.
- Si hay error, javac indica línea y tipo de problema.

Ejecutar desde terminal

Después de compilar, lanzamos la clase

```
java HolaMundo
```

Para ejecutar no se escribe .java ni .class. Se escribe java y el nombre de la clase.

```
Correcto:  java HolaMundo  
Incorrecto: java HolaMundo.java  
Incorrecto: java HolaMundo.class
```

Si aparece `ClassNotFoundException`, revisa que estés en la carpeta correcta y que el nombre coincida.

Errores frecuentes de sintaxis

Los primeros errores suelen repetirse mucho

- Falta un punto y coma al final de una instrucción.
- Falta cerrar una llave, paréntesis o comilla.
- El nombre del archivo no coincide con la clase pública.
- Se escribe main, String o System con mayúsculas incorrectas.
- Se intenta ejecutar desde una carpeta distinta.

```
error: ';' expected
error: reached end of file while parsing
error: cannot find symbol
```

Primero se corrige la sintaxis; después se revisa la lógica.

Cómo leer un error en VS Code

No hay que asustarse: el error da pistas

Cuando Java muestra un error, debemos mirar tres cosas: el archivo, la línea y el mensaje principal.

```
Ejercicio2.java:10: error: class Ejercicio2 is public,  
should be declared in a file named Ejercicio2.java
```

- Archivo: dónde ocurrió.
- Línea: zona aproximada que hay que revisar.
- Mensaje: qué cree Java que pasa.
- A veces el fallo real está una línea antes.

Truco docente: pedir al alumno que lea el error en voz alta antes de tocar nada.

Buenas prácticas de nombres

Nombres claros ayudan a aprender y corregir

```
public class ConversorTemperatura {  
    public static void main(String[] args) {  
        double celsius = 25;  
    }  
}
```

Para clase pública: nombre descriptivo + archivo idéntico.

- Clases: PascalCase → HolaMundo, PrecioConIVA.
- Variables: camelCase → nombreAlumno, precioFinal.
- Evitar nombres como a, b, cosa o prueba1.
- El nombre debe explicar la intención del programa.

Ejemplo guiado: HolaMundo

Programa mínimo para comprobar que todo funciona

```
/*
Enunciado:
Crea un programa que muestre el mensaje Hola, mundo.
*/

public class HolaMundo {

    public static void main(String[] args) {

        System.out.println("Hola, mundo");

    }

}
```

- Crear archivo HolaMundo.java.
- Escribir el código respetando mayúsculas.
- Guardar.
- Compilar con javac HolaMundo.java.
- Ejecutar con java HolaMundo.

Práctica de aula

Tres ejercicios para fijar la sintaxis antes de avanzar

- FichaPersonal: mostrar nombre, ciudad y objetivo del curso.
- PrintVsPrintln: comprobar la diferencia entre print y println.
- DetectaErrores: corregir mayúsculas, String, System y punto y coma.

En este punto todavía no hacen falta variables: lo importante es la estructura.

```
public class FichaPersonal {  
    public static void main(String[] args) {  
        System.out.println("Nombre: Ana");  
        System.out.println("Ciudad: Gijón");  
        System.out.println("Objetivo: aprender  
Java");  
    }  
}
```

Resumen antes de variables

Lista rápida de comprobación

- Sé crear un archivo .java con el nombre correcto.
- Reconozco public class y el método main.
- Uso llaves, paréntesis, comillas y punto y coma.
- Muestro texto con System.out.print() y println().
- Puedo leer errores básicos de compilación.

Siguiente paso natural: variables, tipos de datos, constantes y operadores.

```
int edad = 35;  
double precio = 19.95;  
String nombre = "Lucía";  
final double IVA = 0.21;
```